

KUBERNETES

SERVICES

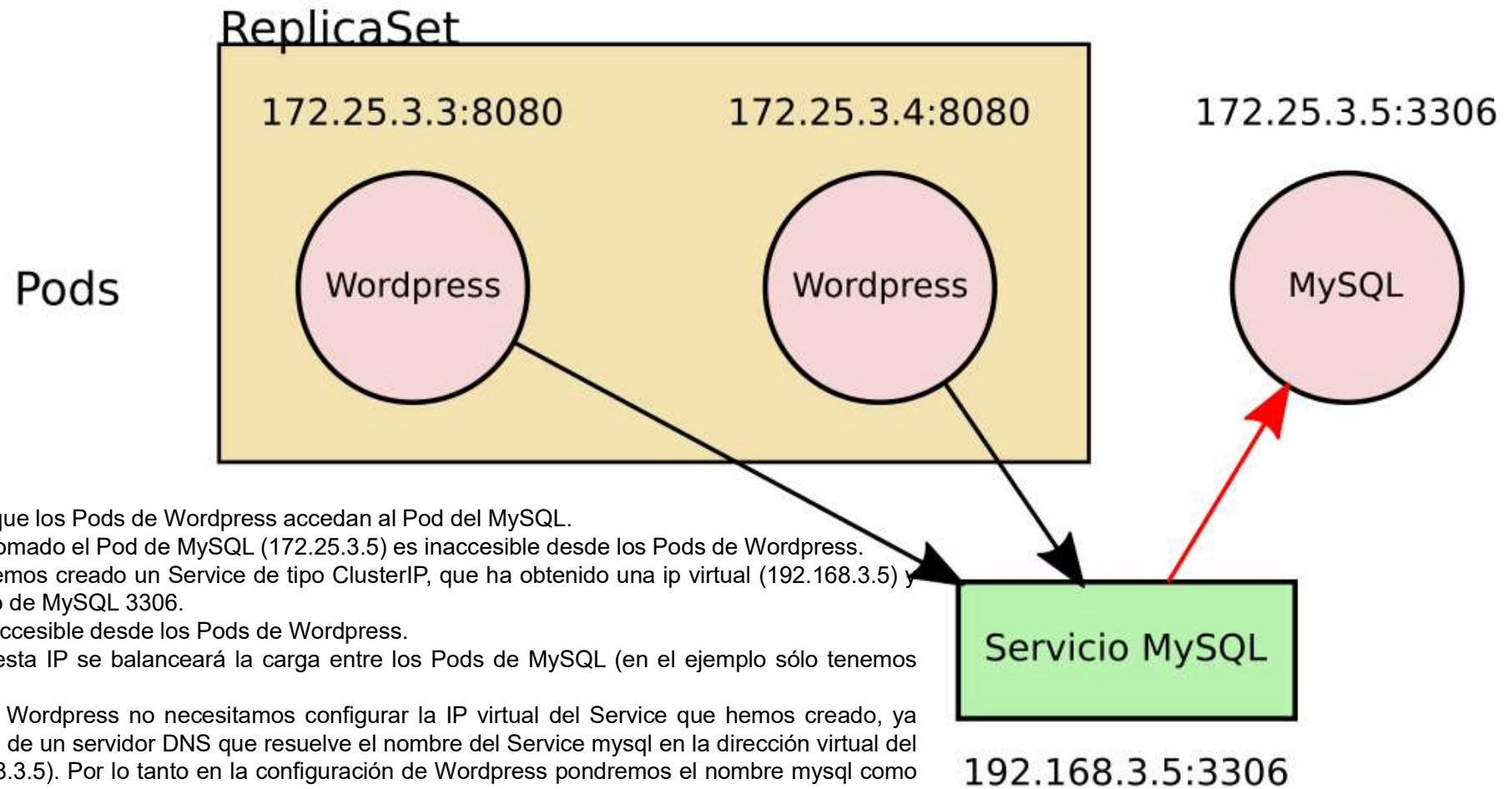
KUBERNETES

Los servicios ([Services](#)) nos permiten acceder a las aplicaciones que hemos desplegado en el cluster.

- Un Service es una abstracción que **nos permite acceder a un conjunto de pods** (que se han creado a partir de un Deployment) que implementan una aplicación (Por ejemplo: acceder a un servidor web, a una servidor de base de datos, a un servicio que forma parte de una aplicación, ...).
- A cada Pod se le asigna una IP a la que no se puede acceder directamente, por lo tanto necesitamos un Service que nos ofrece **una dirección virtual (CLUSTER-IP) y un nombre** que identifica al conjunto de Pods que representa, al cual nos podemos conectar.

KUBERNETES

ClusterIP

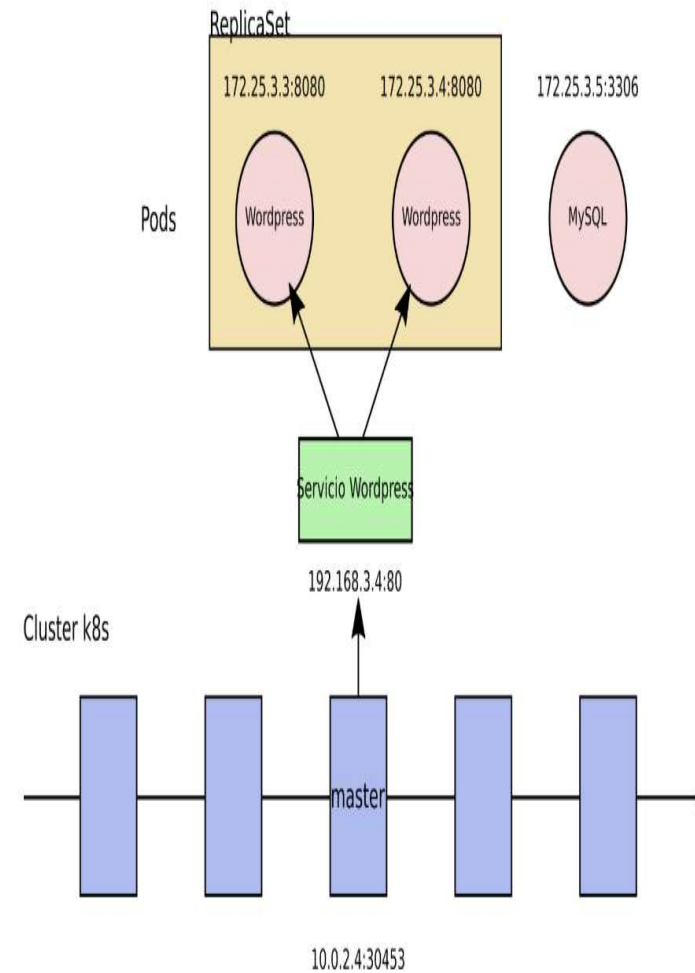


- 1.Necesitamos que los Pods de Wordpress accedan al Pod del MySQL.
- 2.La IP que ha tomado el Pod de MySQL (172.25.3.5) es inaccesible desde los Pods de Wordpress.
- 3.Por lo tanto hemos creado un Service de tipo ClusterIP, que ha obtenido una ip virtual (192.168.3.5) y expone el puerto de MySQL 3306.
- 4.Esta IP sí es accesible desde los Pods de Wordpress.
- 5.Al acceder a esta IP se balanceará la carga entre los Pods de MySQL (en el ejemplo sólo tenemos uno).
- 6.Además en el Wordpress no necesitamos configurar la IP virtual del Service que hemos creado, ya que disponemos de un servidor DNS que resuelve el nombre del Service mysql en la dirección virtual del Service (192.168.3.5). Por lo tanto en la configuración de Wordpress pondremos el nombre mysql como host del servidor de base de datos al que debe acceder.

KUBERNETES

NodePort

1. Necesitamos que los Pods de Wordpress sean accesibles desde el exterior, para que podamos acceder a la aplicación.
2. La IP que han tomado los Pods de Wordpress (172.25.3.3, ...) no son accesibles desde el exterior. Además comprobamos que estos Pods están ofreciendo el servicio en el puerto 8080.
3. Por lo tanto, hemos creado un Service de tipo NodePort que ha obtenido una IP virtual (192.168.3.4) y expone el puerto 80.
4. Al acceder a esta IP al puerto 80 se balanceará la carga entre los Pods de Wordpress, accediendo a las IPs de los Pods de Wordpress al puerto 8080.
5. El Service NodePort ha asignado un puerto de acceso aleatorio (entre el 30000 - 40000) que nos permite acceder a la aplicación mediante la IP del nodo master (minikube ip). En el ejemplo si accedemos a 10.0.2.4:30453 estaremos accediendo al Service que nos permitirá acceder a la aplicación



KUBERNETES

LoadBalancer

LoadBalancer

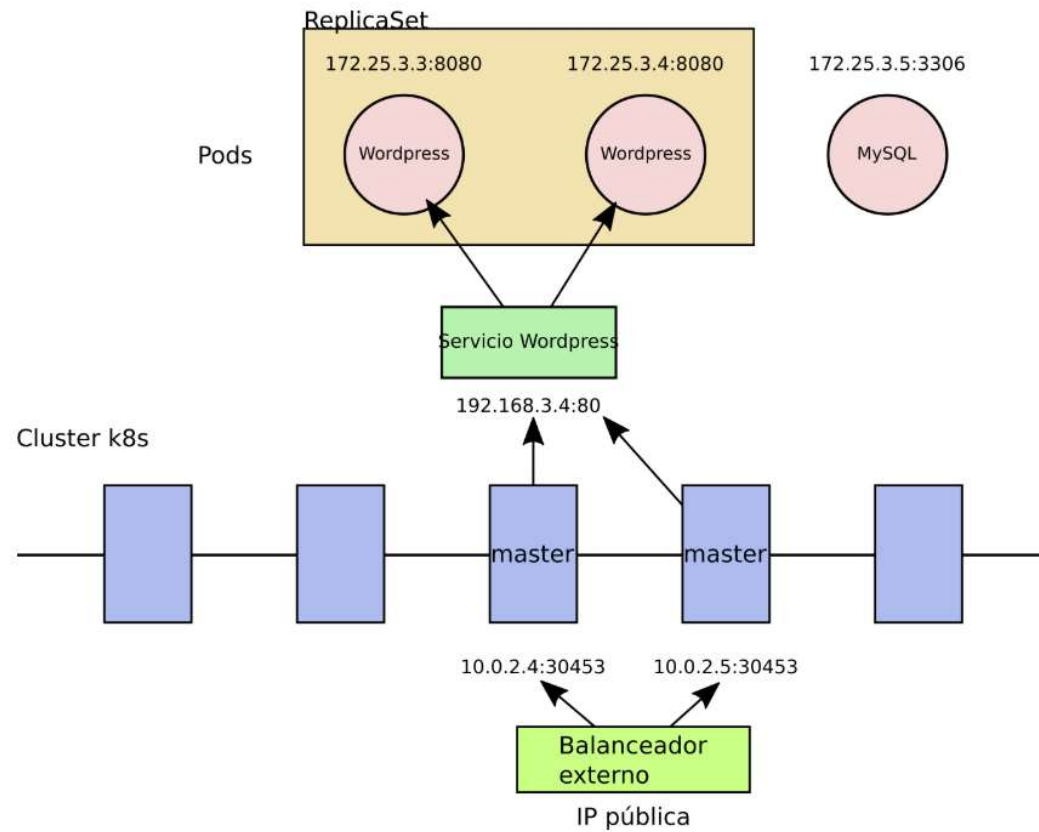


Imagen de elaboración propia. Servicio LoadBalancer (CC BY-NC-SA)

KUBERNETES

```
apiVersion: v1
kind: Service
metadata:
  name: nginx
spec:
  type: NodePort
  ports:
  - name: service-http
    port: 80
    targetPort: http
  selector:
    app: nginx
```

Veamos la descripción:

- Vamos a crear un recurso Service (parámetro kind) y lo nombramos como nginx (parámetro name). Este nombre será importante para la resolución dns.
- En la especificación del recurso indicamos el tipo de Service (parámetro type).
- A continuación, definimos el puerto por el que va a ofrecer el Service y lo nombramos (dentro del apartado port: el parámetro port y el parámetro name). Además, debemos indicar el puerto en el que los Pods están ofreciendo el Service (parámetro targetPort), en este caso, hemos usado el nombre del puerto (http) que indicamos en el recurso Deployment:
... ports: - name: http containerPort: 80 ...
- Por ultimo, seleccionamos los Pods a los que vamos acceder y vamos a balancear la carga seleccionando los Pods por medio de sus etiquetas (parámetro selector).

KUBERNETES

Crear directamente el Servicio desde el fichero yaml:	<code>kubectl apply -f servicio.yaml</code>
Ver los recursos que se han creado:	<code>kubectl get all</code>
Si queremos obtener información detallada del recurso Service que hemos creado:	<code>kubectl describe service nombre</code>
Si queremos ver los logs generados en los Pods de un service	<code>kubectl logs service/nombre</code>
Si eliminamos el service se eliminarán el deployment asociado	<code>kubectl delete service nombre</code>

KUBERNETES

Hasta ahora tenemos dos opciones principales para acceder a nuestras aplicaciones desde el exterior:

1.Utilizando Services del tipo NodePort: Esta opción no es muy viable para entornos de producción ya que tenemos que utilizar puertos aleatorios desde 30000-40000.

La solución puede ser utilizar un [Ingress controller](#) que nos permite utilizar un proxy inverso (HAproxy, nginx, traefik,...) que por medio de reglas de encaminamiento que obtiene de la API de Kubernetes, nos permite el acceso a nuestras aplicaciones por medio de nombres.

2.Utilizando Services del tipo LoadBalancer: Esta opción sólo es válida si trabajamos en un proveedor Cloud que nos ofrece un balanceador de carga para cada una de las aplicaciones, en cloud público puede ser una opción muy cara.